

Enterprise Jenkins MCP

Architecture, security, performance, and implementation plan

A production blueprint for evolving `simonfxr/go-jenkins-mcp` into a per-user, locally hosted and gateway-capable enterprise integration.

Starting point	simonfxr/go-jenkins-mcp
Primary client	Cursor, local stdio MCP
Optional deployment	AgentCore / managed gateway near Jenkins
Prepared	July 31, 2026
Revision	Engineer authentication, policy, and seekable-storage update
Status	Revised target architecture and delivery plan

Core decision: Jenkins is not modeled as a native three-legged OAuth authorization server. The preferred browser path uses an external IdP and Jenkins-audience bearer tokens; local personal API tokens remain supported through the operating-system credential store.

Contents

The document is structured as a decision and implementation reference. Word navigation uses the heading hierarchy.

1	Executive recommendation
2	Engineer-note validation and resulting decisions
3	Product goals and non-goals
4	Assessment of the starting repository
5	Target architecture
6	Authentication and identity
7	Global read-only and MCP-side RBAC
8	Network and bandwidth architecture
9	Tiered log and artifact storage
10	Ratarmount-rs and seekable Zstandard integration
11	Enterprise feature inventory
12	MCP tool design
13	Diagnostics and search engine
14	Security and privacy design
15	Performance engineering plan
16	Reliability and data integrity
17	Observability and supportability
18	Testing strategy
19	Packaging, deployment, and lifecycle
20	Delivery roadmap
21	Release gates
22	Key design decisions to approve
23	Open questions requiring owner decisions
24	Source and technology notes
25	Bottom line

1. Executive recommendation

Use `simonfxr/go-jenkins-mcp` as a **behavioral seed**, not as the long-term architecture. It already proves the basic Go/MCP/Jenkins integration path and supplies useful compatibility fixtures. Before adding enterprise behavior, split the monolith into testable packages, freeze its current external behavior, and correct the log path so a bounded request cannot cause an unbounded Jenkins transfer or `io.ReadAll` allocation.

The product should support two deployment shapes without changing its tool contracts:

1. **Local per-user stdio mode.** Cursor starts one local process. Personal API tokens or OAuth refresh material stay in the operating-system credential store. No local listening port is needed except a temporary loopback OAuth callback.
2. **Managed gateway mode.** A security-approved MCP deployment can sit near Jenkins to avoid VPN transfer costs. It must still operate as the individual user, preserve subject identity end to end, isolate cache data by tenant/profile/user policy, and never fall back to a generic service account.

The recommended enterprise design is:

1. **Per-user identity only.** The supported initial credential path is a personal Jenkins API token in the OS credential store. The preferred browser path uses Entra ID or another approved external authorization server to issue a Jenkins-audience access token. Jenkins validates that token as an OAuth resource server.
2. **Do not describe Jenkins as a native 3LO provider.** Jenkins core does not issue authorization codes or OAuth access tokens for third-party clients. The Jenkins OIDC login realm authenticates browser sessions and directs scripted clients to personal API tokens. Native 3LO is therefore not a feature of the starting Jenkins deployment.
3. **Use the least-complex OAuth architecture that meets the gateway contract.** First prove external IdP + JWT bearer validation. Add AgentCore user-delegated authorization-code 3LO and/or token exchange/on-behalf-of integration as the managed-gateway contract requires. Build a Jenkins plugin only if the gateway contract truly requires Jenkins to expose provider-style consent/token endpoints or a narrow exchange endpoint.
4. **Authorization is an intersection, never a union.** Effective access is `Jenkins permissions AND MCP policy AND global read-only mode AND request limits`. MCP-side RBAC can remove permissions but can never grant access Jenkins denied.
5. **Global read-only is a hard kill switch.** It is configurable from Cursor process arguments/environment, profiles, and signed enterprise policy. A true value wins at every layer. Mutation tools are omitted from discovery and crafted/direct calls are denied again at dispatch and Jenkins-client boundaries.
6. **Download each remote log byte once per generation.** Mirror progressive logs incrementally, compress immediately into independently decodable Zstandard frames/chunks, index them locally, and perform search/diagnosis locally.
7. **Use seekable Zstandard archives for cold storage.** Related logs are packed together into immutable `.tar.zst` volumes using independent Zstandard frames plus a seek table/checkpoint index compatible with `ratarmount-rs`. A normal single-frame `.tar.zst` is not acceptable.
8. **Ratarmount-rs is the preferred L2 engine, not the hot path.** The exact implementation/repository preferred by engineering must be supplied and qualified. Pin the approved production commit or release, then place it behind an `ArchiveStore` interface with a native non-FUSE fallback.
9. **Triage-shaped tools and bounded evidence.** High-level tools should return structured findings and exact evidence handles. Whole logs, archives, and large metadata graphs do not enter model context.

10. **Performance is a release contract.** Network bytes, decompression/read amplification, MCP response bytes, memory, CPU, archive lookup latency, index time, cache quota, and concurrency all receive explicit budgets and regression tests.

Immediate decisions

Decision	Baseline
Local authentication	Personal Jenkins API token in OS credential store
Browser authentication	Authorization Code + PKCE against Entra/approved IdP; Jenkins is resource server
Native Jenkins 3LO	Not available; optional custom plugin/broker only if AgentCore requires it
Default operations	Read-only
Additional authorization	Signed, restricting MCP-side RBAC
Active log format	Independent, line-aware Zstandard frames/chunks
Cold archive format	Immutable seekable multi-frame <code>.tar.zst</code> with TAR member and frame indexes
Archive grouping	Related build/stage/downstream/test logs share a bounded affinity pack
Archive reader	<code>ratarmount -rs</code> preferred, native Go fallback mandatory

2. Engineer-note validation and resulting decisions

The engineer notes are materially relevant and change the authentication and storage decisions. They are incorporated as follows.

2.1 Accepted findings

Engineer finding	Disposition	Architectural consequence
Jenkins core is not a general three-legged OAuth authorization server	Accepted and release-blocking for terminology/design	Do not point OAuth discovery or token endpoints at Jenkins unless a new plugin/broker explicitly implements them
Jenkins scripted clients normally use Basic auth with a personal API token	Accepted	Keep secure local API-token mode as the first production path
<code>oic-auth</code> is a browser security realm, not delegated REST API authorization	Accepted	Do not treat a successful Jenkins browser login as an MCP API token
<code>oidc-provider</code> issues workload identity from builds to external clouds/services	Accepted; not applicable to user-to-Jenkins API access	Exclude it from the client authentication design
<code>github-oauth</code> is a GitHub-specific UI realm	Accepted; not a general solution	Exclude it from the generic enterprise 3LO design
<code>oauth-credentials</code> is a credentials framework, not an OAuth delegation server	Accepted; not a solution	Exclude it from the API-authentication critical path
<code>jwt-auth-filter</code> can validate Jenkins-audience bearer JWTs	Accepted as the closest current resource-server option	Test it with Entra/token exchange and inventory every Jenkins route used by the MCP

Engineer finding	Disposition	Architectural consequence
AgentCore uses per-user identity/token storage for other MCPs	Relevant deployment context	Add a gateway compatibility spike and preserve per-user subject binding; do not copy Atlassian/Bitbucket details into Jenkins code
A custom Jenkins 3LO plugin would close a gateway gap	Conditional strategic item	Treat as an optional, separately security-reviewed project after proving simpler resource-server/token-exchange paths
<code>ratar mount -rs</code> is the preferred archive implementation	Accepted as a product preference; exact dependency not supplied or independently verified	Obtain and audit the exact repository, commit/release, license, API, supply chain, format compatibility, and platform behavior. Keep the format behind ArchiveStore and retain a native Go reader.

2.2 Important qualification of `jwt-auth-filter`

The plugin makes Jenkins a bearer-token **resource server**. It is not an authorization server and does not create a complete native 3LO flow by itself. An external IdP must issue a token with the expected Jenkins audience, or a broker must exchange the user's gateway token for such a token.

The plugin also requires hardening work before enterprise use:

- Invalid or missing bearer tokens may fall through to other Jenkins authentication filters; an OAuth-required profile must prove that protected routes cannot silently use Basic, API-token, session-cookie, or anonymous access.
- The default protected pattern focuses on `/**/api/**`, while this MCP also uses non-`api` routes such as progressive console text, artifact download, queue/build controls, and plugin-specific Pipeline/test endpoints. Every route must be inventoried and protected.
- Claim mapping, group overage, JWKS behavior/caching, key rotation, token exchange, and revocation behavior must be tested against the approved Entra application and Jenkins authorization strategy.
- The plugin's `/mcp/**` example applies to a Jenkins-hosted MCP endpoint. A local sidecar calls Jenkins REST endpoints, so protecting only `/mcp/**` would not protect this design.

2.3 AgentCore relevance

The existing Bitbucket/Jira/Confluence flow is useful as a **security and identity pattern**: the gateway receives an Entra-authenticated user, retrieves or exchanges a per-user downstream credential, stores refresh material in a vault keyed to that user, and avoids shared service identities. The implementation details for VPC Lattice, Atlassian on-premises OAuth, Bitbucket Cloud consent, and Jira automation PATs are not Jenkins MCP requirements.

For Jenkins, evaluate these paths in order:

1. **AgentCore user-delegated 3LO against Entra:** Configure an AgentCore `Custom0auth2` or approved Microsoft provider whose discovery, authorization, and token endpoints are Entra endpoints and whose resource/audience is the dedicated Jenkins API. AgentCore performs the authorization-code consent flow and stores/refreshes the resulting credential under the user-workload binding.
2. **On-behalf-of or token exchange:** Exchange the authenticated caller's gateway token for a short-lived Jenkins-audience token through Entra using the approved OBO, RFC 8693, or RFC 7523 profile. This is preferable where consent/preauthorization and tenant policy support it.

3. **Direct bearer pass-through:** Forward an inbound token only when it was already issued for the exact Jenkins resource, the gateway and Jenkins independently validate it, and security approves preserving that audience end to end. Never forward a generic gateway or Graph token.
4. **Narrow broker/plugin:** A small approved component maps a verified subject to a short-lived Jenkins credential/token without becoming a full OAuth platform.
5. **Full Jenkins 3LO authorization-server plugin:** Last resort only. Consent, client registration, redirect validation, token issuance, rotation, revocation, scopes, audit, and secure storage would become security-critical Jenkins responsibilities.

2.4 What is not a core product requirement

The document retains the engineer's other MCP examples as background, but does not make these Jenkins deliverables: Bitbucket Cloud-specific 3LO behavior, Jira/Confluence VPC Lattice routing, AgentCore's internal vault implementation, or a Jira automation PAT provider. The Jenkins MCP needs stable interfaces for identity propagation and token acquisition, not hard dependencies on those product-specific mechanisms.

2.5 Storage terminology correction

The requirement is **Zstandard random access through independent frames/checkpoints**, not arbitrary internal Zstandard blocks. Frames are independently decodable; compressed blocks inside one frame can depend on previous blocks. The archive writer must therefore create multiple bounded frames and a seek table/index. The phrase "block size" may remain as a user-facing tuning term, but the persisted format and tests must speak in terms of independent frames and uncompressed checkpoints.

3. Product goals and non-goals

3.1 Goals

- Give Cursor fast, safe, per-user access to Jenkins jobs, builds, stages, tests, logs, artifacts, agents, queues, and health data.
- Diagnose failures with small, structured responses backed by precise evidence ranges.
- Minimize bytes transferred from Jenkins and bytes returned through MCP.
- Keep long-lived credentials and refresh tokens local to the user and outside Cursor configuration.
- Support personal Jenkins API tokens, external-IdP OAuth bearer tokens, and an optional managed gateway mode without shared identities.
- Work with multiple Jenkins controllers through separately isolated profiles.
- Operate efficiently against very large logs and long build histories.
- Preserve useful cached evidence while respecting per-user/profile isolation, MCP RBAC, quotas, retention, sensitivity, revocation, and legal/security policy.
- Degrade gracefully when Jenkins plugins or optional APIs are absent.
- Package as a signed, easy-to-deploy Windows binary first, with Linux and macOS support where required.
- Produce enough audit and diagnostic data for security and platform teams without logging secrets or full customer data.

3.2 Non-goals for the first production release

- Granting access that Jenkins denied. MCP-side RBAC is intentionally restricting only.
- Scraping or replaying SAML browser session cookies.

- Allowing an LLM to edit Jenkins job configuration, credentials, plugins, nodes, or controller settings.
- Sending complete multi-gigabyte logs to the model.
- Requiring a central shared MCP service or shared service account. A managed gateway remains optional and per-user.
- Treating FUSE as a required runtime dependency for normal log access. `ratarmount-rs` must also support a library/sidecar or direct archive API.
- Building a full Jenkins OAuth authorization server in the first release. External IdP/resource-server and token-exchange paths are evaluated first.
- Building an embeddings or vector-search platform before literal and regular-expression search are proven insufficient.
- Running arbitrary downloaded artifacts or build scripts on the developer workstation.
- Guaranteeing semantic root-cause analysis without presenting the underlying evidence and confidence limits.

4. Assessment of the starting repository

4.1 Useful starting capabilities

The seed repository already provides:

- A Go executable with MCP stdio support and optional Streamable HTTP.
- Jenkins job listing and job detail.
- Running builds and queue information.
- Build metadata and parameters.
- Console log access through Jenkins `progressiveText`.
- Build log tail retrieval.
- Build triggering, stopping, queue waiting, and build waiting.
- Search across previous builds by result and parameters.
- Nested Jenkins folder path handling.
- Jenkins crumb acquisition for applicable write operations.
- Request cancellation through Go contexts in portions of the call path.

These are useful compatibility fixtures. Preserve their externally useful semantics where practical, but do not preserve internal coupling.

4.2 Enterprise gaps and risks

Area	Current condition	Enterprise requirement
Architecture	Most behavior in one large <code>main.go</code>	Separate transport, MCP, Jenkins client, auth, storage, indexing, tools, policy, and observability packages
Credentials	<code>user:token</code> CLI flag or environment variable	OS credential store, no secret CLI arguments, profile isolation, login/logout/status commands
OAuth	Not present	External IdP Authorization Code with PKCE, Jenkins-audience token, resource-server validation, gateway compatibility; no false native-3LO claim
Authorization policy	Mutating tools available with basic configuration	Global read-only kill switch plus signed restricting MCP RBAC enforced at registry, dispatch, service, and client layers

Area	Current condition	Enterprise requirement
Log transfer	Response can be read before requested truncation	Incremental progressive reads with hard byte caps and no hidden over-download
Log storage	None	Immediate multi-frame Zstandard compression, line indexes, seekable related-log TAR packs, quota, retention, recovery
Search	Raw slices and simple build search	Local literal/RE2 search, context windows, first-error and signature extraction
Pipelines/tests	Limited build-level view	Stage graph, stage logs, JUnit, flaky tests, last-green comparison
Build graph	No robust traversal	Upstream/downstream and Pipeline-trigger traversal with cycle and fan-out limits
Artifacts	Not exposed	Safe metadata, selective download, bounded inspection, archive-bomb protection
Controller health	Minimal	Nodes, executors, queue pressure, plugin/controller capabilities and health
Networking	Basic timeouts	Connection reuse, HTTP/2, custom CAs, proxy, mTLS, redirect protection, retry policy
Response efficiency	Tool-specific behavior	Global default and hard output budgets, pagination handles, stable schemas
Security	No comprehensive redaction or content controls	Secret redaction, terminal escape sanitization, prompt-injection labeling, secure cache ACLs
Quality	Small project with no broad enterprise suite	Unit, integration, conformance, compatibility, load, chaos, recovery, and security tests
Distribution	Local build	Signed local packages plus optional AgentCore/managed-gateway deployment, SBOM, provenance, scans, and controlled updates

4.3 First corrective action

Create an internal fork and freeze a reproducible baseline. Then split the implementation before adding significant behavior:

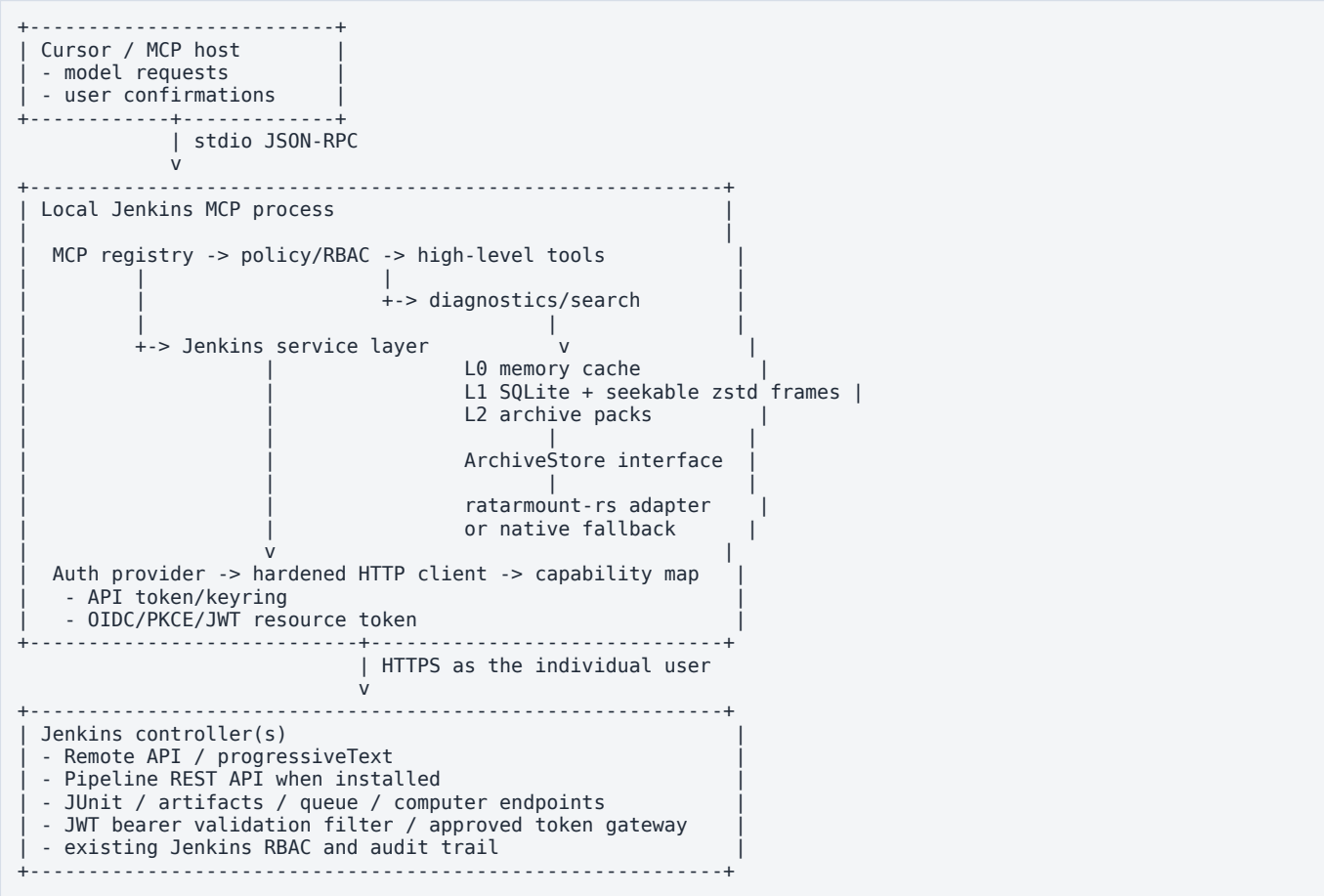
```
cmd/jenkins-mcp/
internal/app/
internal/config/
internal/profile/
internal/auth/
internal/keyring/
internal/jenkins/
internal/capabilities/
internal/mcpserver/
internal/tools/
internal/policy/
internal/logmirror/
internal/store/
```



```
internal/archive/  
internal/search/  
internal/diagnostics/  
internal/redact/  
internal/audit/  
internal/telemetry/  
internal/update/  
pkg/contracts/      # only if stable public contracts are needed
```

The Jenkins client must not import MCP types. Tool handlers must not construct raw HTTP requests. Storage and archive implementations must be replaceable through narrow interfaces. This separation is the main condition for using the seed repository successfully.

5. Target architecture



5.1 Runtime modes

Normal mode: local stdio only. Cursor starts and stops the process. This is the default and preferred deployment.

Local diagnostic HTTP mode: optional, disabled by default, loopback-only, authenticated, origin-protected, body-limited, and intended for integration testing or multiple approved local clients.

Managed AgentCore/gateway mode: optional after the local pilot. The gateway uses the same tool and policy contracts but runs near Jenkins. It accepts only an authenticated individual subject, obtains a per-user

Jenkins credential/token through an approved provider, partitions cache/audit state, and propagates a correlation ID. It must not translate all users into one Jenkins service identity.

Headless automation mode: optional later and separate from interactive MCP use. It must not reuse a person's refresh token silently. Use a separately approved workload identity, tool allowlist, and audit policy.

5.2 Core interfaces

```
type CredentialProvider interface {
    Authenticate(ctx context.Context, profile Profile) (AuthSession, error)
    Status(ctx context.Context, profile Profile) (AuthStatus, error)
    Logout(ctx context.Context, profile Profile) error
}

type JenkinsClient interface {
    Do(ctx context.Context, req Request) (Response, error)
    Capabilities(ctx context.Context) (CapabilitySet, error)
}

type LogStore interface {
    Append(ctx context.Context, key LogKey, segment Segment) error
    ReadRange(ctx context.Context, key LogKey, start, length int64) (ReadResult, error)
    Search(ctx context.Context, query SearchQuery) (SearchResult, error)
    Seal(ctx context.Context, key LogKey) error
}

type ArchiveStore interface {
    PutPack(ctx context.Context, pack PackDescriptor) error
    OpenEntry(ctx context.Context, ref ArchiveRef) (io.ReadCloser, EntryMetadata, error)
    Verify(ctx context.Context, ref ArchiveRef) error
    DeletePack(ctx context.Context, ref ArchiveRef) error
}

type PolicyEvaluator interface {
    Evaluate(ctx context.Context, subject Subject, action Action, target Target, attrs Attributes) (Decision, error)
}

type ReadOnlyGate interface {
    Enabled() bool
    DenyMutation(action Action) error
}
```

Interfaces should remain small. Avoid a single universal storage or Jenkins API interface with dozens of unrelated methods.

5.3 End-to-end identity and authorization path

```
verified OS user / gateway subject
    AND
verified Jenkins principal
    AND
Jenkins authorization strategy
    AND
signed MCP-side role/policy
    AND
global read-only + operation budgets
    =
effective MCP permission
```

A failure or unknown state at any term fails closed. Cached policy decisions must be bound to subject, profile, policy version, action, target, and a short revocation TTL.

6. Authentication and identity

6.1 Shared-account prohibition

Every profile is bound to a personal identity. The executable and non-secret configuration may be shared; credentials, refresh material, cache namespaces, and audit identity are not. Status output shows only sanitized profile, method, verified principal, expiry, and policy state.

6.2 Personal Jenkins API-token flow

This is the first production authentication path because it matches Jenkins' documented scripted-client model and needs no new Jenkins plugin.

```
jenkins-mcp profile add corp --url https://jenkins.example.com
jenkins-mcp login --profile corp --method api-token
jenkins-mcp status --profile corp
jenkins-mcp logout --profile corp
```

`login` reads username/token from a non-echoing terminal and stores the token in Windows Credential Manager, macOS Keychain, or Linux Secret Service. Secret command-line options are unsupported. Environment-secret compatibility is opt-in, policy-controlled, and omitted from enterprise examples.

The client sends preemptive Basic authentication with `username:api-token` over verified TLS. It never logs headers. Identity is verified immediately against an approved Jenkins identity endpoint; anonymous fallback or an unexpected principal fails closed.

6.3 Jenkins does not natively provide the required 3LO flow

Jenkins core is a protected API, not a general OAuth authorization server for third-party applications. It does not natively provide the authorization endpoint, consent model, client registration, authorization-code issuance, token endpoint, refresh/revocation contract, and delegated scopes expected by a three-legged OAuth integration.

The existing plugin categories do not change that conclusion:

- `oic-auth` logs people into the Jenkins web UI and recommends API tokens for non-front-end clients.
- `oidc-provider` issues workload identity from Jenkins builds to external services; it is the opposite direction.
- `github-oauth` is a GitHub-specific login realm.
- `oauth-credentials` helps plugins represent OAuth credentials; it does not make Jenkins a delegated authorization server.

The architecture and user documentation must never label ordinary Jenkins OIDC UI login as MCP 3LO.

6.4 Preferred local OAuth path: external IdP + Jenkins resource server

Use Authorization Code with PKCE against Entra ID or another approved IdP. Register the local MCP as a public client and Jenkins as a dedicated API/resource audience. Cursor's local `studio` transport is configured with `command`, `args`, and `env`; the Jenkins MCP itself owns the browser/loopback PKCE flow and OS-keyring persistence. Do not expect Cursor's remote-server OAuth handling to turn Jenkins UI SSO into an API credential.

1. Enterprise profile pins issuer, authorization endpoint, token endpoint, client ID, Jenkins audience/resource, scopes, and redirect policy.

2. MCP generates state, nonce, PKCE verifier, and S256 challenge.
3. MCP binds an ephemeral loopback callback and opens the system browser.
4. User completes corporate login, MFA, and Conditional Access at the IdP.
5. MCP exchanges the code, validates all token properties, keeps the access token in memory, and stores refresh material only in the OS keyring when policy permits.
6. MCP sends `Authorization: Bearer <Jenkins-audience-token>` to Jenkins.
7. Jenkins validates the JWT through `jwt-auth-filter`, a hardened fork, or an approved reverse proxy/plugin and maps the personal subject/groups to Jenkins authorization.

```

Cursor -> local MCP -> Entra authorization code + PKCE
      |
      +--> Jenkins-audience JWT
            |
            v
        Jenkins JWT resource filter
            |
            v
        personal Jenkins principal/RBAC

```

A Graph token, generic gateway token, ID token, or token for another audience is never accepted as a Jenkins API credential.

6.5 Jenkins route coverage and anti-fallback requirements

OAuth acceptance must be tested against an explicit route manifest, not only `/**/api/**`. At minimum inventory and protect:

- Controller, folder, job, view, build, queue, node, and identity JSON/XML endpoints.
- `logText/progressiveText` and any progressive HTML/text variants.
- Pipeline stage/node APIs and stage-log endpoints.
- JUnit/test report endpoints.
- Artifact listing and artifact-byte download paths.
- Crumb endpoint where used.
- Build trigger, parameterized build, queue cancellation, stop/term/kill/replay endpoints when mutations are enabled.
- Plugin-specific endpoints discovered during capability qualification.

For an OAuth-required profile, each route must demonstrate all of the following: valid token maps to expected person; wrong audience/issuer/expiry fails; missing/invalid bearer cannot fall through to Basic, API-token, UI-session, or anonymous access; and audit data shows the person. A reverse proxy or plugin patch may be needed because the current filter intentionally lets failed JWT authentication continue to other filters.

6.6 AgentCore/managed gateway authentication

The gateway receives an Entra-authenticated personal subject. It must then acquire a **per-user Jenkins credential** through a provider contract:

```

type DownstreamCredentialProvider interface {
    Acquire(ctx context.Context, subject Subject, resource Resource, scopes []string) (ShortLivedCredential, error)
    Revoke(ctx context.Context, subject Subject, resource Resource) error
}

```

Qualification order:

1. Configure AgentCore user-delegated authorization-code flow against Entra for the Jenkins resource. The authorization-server endpoints are Entra's, not Jenkins', and AgentCore's token vault binds the downstream credential to the validated user and workload identity.
2. Prototype Entra on-behalf-of, RFC 8693, or RFC 7523 exchange to mint a short-lived Jenkins-audience token from the authenticated caller context.
3. Permit direct bearer pass-through only when the inbound access token already has the exact Jenkins audience and the end-to-end trust model is approved.
4. Use a narrow internal broker/plugin to exchange a verified subject for a short-lived Jenkins credential when Entra cannot issue the required resource token directly.
5. Build a complete Jenkins 3LO authorization-server plugin only if the AgentCore provider contract cannot support the first four and a funded security-owner decision explicitly approves it.

Tokens and refresh material are vault/keyring stored under the actual user subject. The gateway is not allowed to cache one user's credential under another identity or substitute a shared account. The local Cursor path and managed AgentCore path remain separate providers sharing the same identity and policy contracts.

6.7 Conditional custom Jenkins plugin scope

A custom plugin is a separate security product, not a minor MCP feature. Prefer a **narrow resource filter or token-exchange endpoint** over a full OAuth authorization server. If full 3LO is required, the plugin backlog must include:

- OAuth/OIDC discovery and protected-resource metadata.
- Client registration governance and redirect URI validation.
- Authorization endpoint, authenticated consent, scopes, state/PKCE, and anti-CSRF.
- Authorization-code single use, token issuance, signing keys, rotation, and JWKS.
- Refresh-token rotation, revocation, expiry, replay detection, and audit.
- Mapping of Entra subject/groups to Jenkins user and authorization strategy.
- HA/session behavior, backup/restore, upgrade compatibility, penetration testing, and incident response.

No plugin-generated long-lived token may become a disguised shared credential.

6.8 jwt-auth-filter qualification items

Before production, review or patch:

- Claim-name configurability and group/role normalization.
- Entra group overage and large-group resolution.
- JWKS caching, outage behavior, refresh, and key rotation.
- Exact protected-path semantics and coverage of non-API Jenkins routes.
- Fail-closed behavior for missing, malformed, expired, or wrong-audience tokens.
- Multiple issuers/audiences and controller isolation.
- Token exchange (`act`, `azp`, `scp`/roles, subject) validation rules.
- Performance under concurrent local/gateway clients.

6.9 SAML

SAML can remain behind corporate browser SSO, but the MCP must not scrape/replay Jenkins browser cookies or automate SAML form posts. Use an approved OIDC bridge, external IdP, or token broker.

6.10 Multiple controllers and credential isolation

Each profile has an immutable normalized origin, auth provider, credential-store namespace, OAuth audience, TLS/proxy settings, cache namespace, policy version, quotas, and retention. Tools resolve exactly one profile unless a cross-profile read tool explicitly aggregates source-labelled results. Tokens are never forwarded across origins, redirects, profiles, or controller aliases.

7. Global read-only and MCP-side RBAC

7.1 Global read-only kill switch

Read-only is a runtime safety boundary, not merely documentation or a hidden-tool convention. The effective value is computed from all inputs using **most restrictive wins**:

```
built-in default (true for pilot)
OR signed enterprise force_read_only
OR profile read_only
OR process --read-only
OR JENKINS_MCP_READ_ONLY=true
OR emergency safe mode
```

No user-controlled setting can turn off a signed enterprise `force_read_only`. Conflicting values do not produce an ambiguous state: any true value makes the process read-only. The pilot exposes no generic `--read-write` inverse switch; enabling mutations requires an explicit profile/policy capability and the separate mutation framework.

Recommended Cursor configuration:

```
{
  "mcpServers": {
    "jenkins": {
      "command": "C:\\Tools\\jenkins-mcp.exe",
      "args": ["serve", "--profile", "corp", "--read-only"],
      "env": {
        "JENKINS_MCP_READ_ONLY": "true"
      }
    }
  }
}
```

The CLI flag is the preferred visible control. The environment value is supported for managed configuration. `status` shows the effective value and all non-secret contributing sources.

When read-only is active:

- Mutating tools are not registered or advertised.
- A crafted JSON-RPC invocation of a disabled tool is rejected.
- Service methods reject mutation actions even if called internally.
- Jenkins transport rejects unsafe HTTP methods/routes unless explicitly classified as a read exception.
- Background jobs cannot trigger mutations.
- Audit records identify the denial and policy source.

7.2 MCP-side restricting RBAC

Jenkins RBAC remains authoritative for what Jenkins will allow. The MCP adds a more restrictive layer for tool exposure, sensitive jobs, expensive operations, cache behavior, and optional mutations.

```
EffectivePermission = JenkinsPermission
                     AND GlobalReadOnlyGate
                     AND MCPRolePolicy
                     AND Controller/Job/Artifact Constraints
                     AND OperationBudgets
```

The MCP policy can deny a Jenkins administrator access to a sensitive MCP feature; it can never make an inaccessible Jenkins object available.

7.3 Policy subjects and trust

Local mode binds policy to the verified Jenkins identity and, when OAuth is used, the validated external subject/tenant. Managed mode binds it to the gateway-authenticated Entra subject plus the verified Jenkins principal. Never trust an arbitrary username or group supplied as an MCP tool argument, environment variable, unsigned profile, or model text.

7.4 Policy capabilities

The optional RBAC engine should restrict:

- Tool/action class: metadata read, log read, log search, artifact list/download/inspect, diagnostics, health, trigger, stop, replay, administration.
- Controller/profile and folder/job/build target patterns.
- Build parameter names, value patterns, and secret/file parameter use.
- Artifact path, type, size, expansion ratio, and executable-content handling.
- Maximum remote bytes, local bytes scanned, result bytes, concurrency, duration, and history depth.
- Cache retention, pin/export/support-bundle actions, and cross-profile aggregation.
- Sensitive-job labels, customer-data classes, time windows, network location, and deployment mode.
- Mutation confirmation requirements and approver/role conditions.

Default for unknown tools/actions is deny. Policies are versioned, signed, schema-validated, atomically replaced, and fail closed when required policy is missing/expired/invalid.

7.5 Enforcement architecture

Use one policy decision point and multiple policy enforcement points:

1. Registry construction removes unavailable tools.
2. JSON-RPC dispatch evaluates subject/action/target before handler execution.
3. Service methods re-evaluate normalized targets and expanded operations.
4. Jenkins request builder validates method/path/action classification.
5. Storage/export/download layers enforce data and byte policies.
6. Mutation executor requires a bound, expiring confirmation after authorization.

A policy bypass test must attempt direct handler calls, crafted MCP calls, aliases, continuation handles, redirects, background work, and race conditions.

7.6 Policy decision caching and audit

Cache only normalized decisions bound to subject, tenant, profile, policy hash, action, target hash, and short expiry. Invalidate on policy update, logout, identity change, role/group refresh, controller change, or emergency read-only activation. Audit permits and denials without logging log contents, secrets, or sensitive raw parameters.

8. Network and bandwidth architecture

8.1 Governing rule

For logs and other large immutable payloads, in local or managed-gateway mode, the normal steady-state goal is:

Download each byte from Jenkins at most once per local cache generation, then satisfy repeated reads, searches, comparisons, and diagnosis locally.

Exceptions must be observable: cache loss, Jenkins log rewrite/truncation, explicit refresh, retention eviction, or integrity failure.

8.2 Hardened HTTP transport

Create one shared, tuned `http.Transport` and `http.Client` per profile rather than constructing clients per request. Required behaviors:

- TLS 1.2 minimum unless corporate policy requires TLS 1.3.
- Normal certificate verification; explicit custom CA bundle support.
- Optional corporate proxy with `NO_PROXY`-equivalent profile policy.
- Optional mTLS certificate from the OS certificate store or protected file reference.
- HTTP/2 where the controller/reverse proxy supports it.
- Connection pooling and keep-alive with bounded idle connections per host.
- Separate dial, TLS handshake, response-header, idle, and whole-operation timeouts.
- Context cancellation propagated through all requests, readers, searches, and waits.
- Response body size limits before decoding JSON or XML.
- Profile-controlled HTTP content compression for large text responses. Request `gzip` as the conservative baseline when the Jenkins/reverse-proxy path supports it; qualify HTTP `zstd` or Brotli only with explicit end-to-end compatibility evidence. Count encoded wire bytes before decoding, stream-decode directly into the Zstandard chunk writer, and never stage a complete decoded copy.
- Separate encoded-wire and decoded-body limits to resist compressed-response bombs. Benchmark controller/proxy CPU and allow compression to be disabled per endpoint/profile when it increases total cost.
- Redirect policy that refuses cross-origin redirects for authenticated requests and strips credentials on any allowed redirect.
- Canonical origin pinning. Jenkins-provided absolute URLs are accepted only when they match the configured origin or an explicit allowlist.
- Retry only safe/idempotent operations, using jittered exponential backoff and `Retry-After`. Never automatically retry a build trigger unless an idempotency strategy proves the first attempt did not enqueue.
- Per-controller and per-user rate limits, concurrency limits, network-byte budgets, and circuit breakers.
- Gateway mode deduplicates/coalesces only where authorization and cache isolation prove results are identical for each subject.
- Request coalescing for identical metadata fetches.
- Sanitized error messages with Jenkins request IDs where available.

8.3 Efficient Jenkins API use

- Use Jenkins `tree` selectors and appropriate `depth` rather than retrieving complete object graphs.
- Cache stable metadata with short, configurable TTLs and conditional requests where endpoints support validators.
- Prefer bulk endpoints only when they transfer less than N individual calls for the actual need.
- Paginate build history and stop scanning as soon as the requested condition is satisfied.
- Cache controller/plugin capability results and invalidate them on version change or explicit refresh.
- Poll queue/build status adaptively: fast initially, then back off; stop immediately on cancellation.
- Add random jitter so many local MCPs do not synchronize against Jenkins.
- Do not poll completed builds.
- Expose freshness metadata in results so the model can decide whether a refresh is necessary.

8.4 Correct progressive log acquisition

The log mirror owns Jenkins `progressiveText` interaction. Tool handlers never call it directly.

Maintain for each log generation:

- Current raw byte offset.
- Jenkins-reported next offset from `X-Text-Size`.
- `X-More-Data` state.
- Build state and completion timestamp.
- Last successful fetch time.
- Optional fingerprint of an initial prefix and recent boundary bytes.
- Generation ID.

Acquisition algorithm:

1. Request `progressiveText?start=<known-offset>`.
2. Stream the response through a strict counting reader into the active chunk writer.
3. Stop at configured transfer bounds; do not `io.ReadAll` an unbounded body.
4. Commit data and the new offset atomically.
5. If the reported offset moves backward, the prefix fingerprint changes, or Jenkins returns inconsistent data, create a new generation and retain/evict the old one according to policy.
6. While a build runs, schedule another incremental poll with adaptive delay.
7. Once complete and no more data remains, seal the generation and build final indexes.

For a user-requested range, first read the local cache. Fetch only the missing suffix needed to make that range available. For a tail request, use the known mirrored size; do not issue a `start=0` request merely to discover the size of a previously mirrored log.

8.5 Network byte accounting

Record per profile and operation:

- Request count.
- Encoded wire bytes received and sent where measurable.
- HTTP-decoded body bytes and logical Jenkins payload bytes.
- Cache hits and misses.
- Duplicate bytes avoided.
- Progressive log bytes fetched.
- Artifact bytes fetched.

- Compressed bytes written, uncompressed logical bytes, and frame/read amplification.
- VPN versus near-source transfer path when deployment telemetry is approved.
- Bytes returned through MCP.
- Retries, redirects, throttles, and cancellations.

Telemetry must use names and hashes that do not expose job parameters or log text by default.

8.6 MCP response budgets

Apply a central response-budget middleware:

- Default structured result target: **64 KiB**.
- Default log evidence per excerpt: **8 KiB**, with a small number of excerpts.
- Default total tool response hard stop: **1 MiB**.
- Every list tool is paginated.
- Every text-returning tool supports stable continuation handles.
- Oversized fields are summarized with counts and retrieval handles, never silently dropped.
- Responses report truncation, returned byte count, total known size, and continuation state.
- High-level diagnosis returns evidence references and bounded excerpts, not copied full logs.

The budgets are configuration policy, not optional guidance.

9. Tiered log and artifact storage

9.1 Storage objectives

The storage design must minimize disk bytes, filesystem object count, decompression work, and repeated Jenkins/VPN traffic while preserving exact evidence ranges. All stored build output is untrusted and potentially sensitive.

9.2 Tiers

Tier	Purpose	Representation	Read path
L0	Active requests and tiny metadata	Bounded memory buffers, single-flight state	Direct
L1	Running/recent logs and indexes	SQLite metadata + immutable independent Zstandard frame/chunk files	Direct Go reader
L2	Sealed related-log packs	Immutable seekable multi-frame <code>.tar.zst</code> + manifest/index	<code>ratar mount -rs</code> or native fallback
L3 optional	Enterprise/object cold storage	Same immutable pack format, range-readable object store	Gateway/archive service only after qualification

L2 is a capacity and inode-count backstop. It is not inserted between every interactive operation and L1.

9.3 Streaming ingestion and immediate compression

Jenkins progressive log responses are streamed through a counting reader, optional line/control sanitizer metadata scanner, and Zstandard frame writer. The raw response is never persisted as a second complete copy.

For each active log generation:

- Accumulate only a bounded raw buffer, initially targeting 4-16 MiB and ending near a newline when possible.
- Finish an independent Zstandard frame/chunk with content checksum and recorded uncompressed/compressed sizes.
- Record raw byte range, line range/checkpoints, generation, frame/checksum, dictionary ID if any, and source freshness in SQLite.
- Commit file and metadata crash-safely before advancing the durable Jenkins offset.
- Keep very long lines and binary contamination bounded; never wait indefinitely for a newline.
- Choose compression level and worker count from measured throughput, latency, and endpoint-protection behavior. Interactive ingestion takes priority over maximum ratio.

The format must not call internal Zstandard blocks independently seekable. Random access begins at an independent frame/checkpoint.

9.4 Related-log affinity groups

When a build or investigation produces many related text logs, batch them into the same L2 archive when doing so improves locality and remains within shard limits. The default affinity key is:

```
profile/controller + root job/build + pipeline execution/investigation generation
```

Eligible members include:

- Root console log.
- Pipeline stage/node logs.
- Downstream and matrix-child console logs discovered from the same root build.
- JUnit/Ginkgo textual reports and normalized test-failure evidence.
- Small SCM/change summaries and diagnostic index files.
- Archive manifest, line maps, evidence maps, signatures, and checksums.

Do not mix users' private exports, different Jenkins controllers, unrelated customer-sensitive classifications, or objects with incompatible retention/encryption policy. If one affinity group exceeds the pack target, split deterministic volumes while retaining a common group ID and relationship manifest.

Grouping improves diagnosis locality and reduces index/file count, but also increases failure blast radius and rewrite cost. The packer therefore uses bounded 4-16 GiB physical shards initially, transactionally verifies them, and never appends after publish.

9.5 Metadata, indexes, and search

SQLite stores only non-secret metadata and references:

- Profile/controller, job, build, stage/test/artifact relationships.
- Log generations, raw offsets, frame/checkpoint ranges, line samples, checksums, and seal state.
- L2 affinity group, pack ID, TAR member, compressed/uncompressed offsets, and ratarmount/native index version.
- Pins, leases, retention class, sensitivity label, policy hash, and maintenance journal.

Literal/regex search streams only relevant L1 frames or L2 ranges. Search indexes contain positions/signatures rather than duplicate complete text. Results return bounded, redacted excerpts with evidence handles.

9.6 Quota, retention, isolation, and encryption

- Per-user/per-profile roots with restrictive ACLs and symlink/junction checks.
- Separate logical and physical byte accounting for L1/L2 and compression ratio.
- Configurable quotas by profile, build result, age, object type, and sensitivity.
- Running, pinned, leased, or active-investigation data is protected from eviction.
- Low-disk emergency mode stops new caching/packing safely and can prune deterministic candidates.
- Logout may remove credentials without deleting cache; policy can require immediate cache purge or cryptographic key revocation.
- Full-disk encryption is assumed where corporate policy provides it. Optional application-level AEAD uses per-profile keyring keys and independently authenticated frames/entries.

9.7 Artifact storage

Artifacts are metadata-only until selected. Downloads are streamed through size/checksum limits and never executed. Large text artifacts may enter the same affinity pack only after policy/type validation. Binary artifacts normally remain separate or are retained only long enough for bounded inspection to avoid polluting log-oriented packs.

10. Ratarmount-rs and seekable Zstandard integration

10.1 Role and ownership

The preferred archive engine is the intended `ratarmount-rs` implementation. The supplied notes did not identify an exact authoritative repository, and public research did not verify one with that exact project name. Treat it as an internal/private or not-yet-specified dependency until the repository and release are supplied. `ARC-000` must obtain, pin, reproduce, audit, and benchmark the production candidate. The MCP still isolates it behind `ArchiveStore` so archive data remains readable if the Rust adapter is unavailable, upgraded, or disabled by policy.

Ratarmount-rs is the **L2 cold/archive access engine**. L1 active data remains native files/SQLite for predictable append, recovery, and low-latency search.

10.2 Required pack format: seekable Zstandard TAR

The baseline pack is an immutable seekable `.tar.zst`:

```
pack-v1.tar.zst
  Zstd frame 0 -> TAR header/member bytes ...
  Zstd frame 1 -> continued TAR stream ...
  ...
  final seek-table/skipable frame

pack-v1.manifest (cataloged atomically; may also be a TAR member)
  pack ID/schema/policy/classification
  affinity group and volume sequence
  TAR member metadata
  uncompressed TAR offsets and lengths
  Zstd frame compressed/uncompressed offsets and checksums
  logical log byte/line ranges and evidence maps
  object checksums and source identity/freshness

ratarmount-rs/native index
  TAR hierarchy/member offsets
  frame/checkpoint seek map
  index-to-pack checksum/version binding
```

Requirements:

- The writer emits many **independent Zstandard frames**, not one huge frame.
- A seek table/checkpoint index maps uncompressed TAR positions to compressed frame offsets.
- Initial uncompressed frame target is 8 MiB, configurable and benchmarked over 1-32 MiB. Large logs span multiple frames.
- Align frame boundaries with TAR member starts when practical, but do not create pathological tiny frames for thousands of small files. Coalesce small related members while retaining exact TAR/member indexes.
- Enable per-frame content checksums plus whole-member/pack checksums.
- Standard sequential `zstd` decompression must still recover the TAR stream; random readers use the seek metadata.
- The archive is published only after frame table, TAR index, manifest, checksums, and sampled reads pass.
- A normal single-frame `.tar.zst` is rejected by format validation.
- The preferred promotion path supports **zero-recompression payload-frame assembly**. For each sealed TAR member, the packer creates a tiny independent frame containing the TAR header, concatenates the already-compressed L1 payload frames in logical order, and creates a tiny padding frame when required. It then adds TAR termination and the final seek table. The log payload is never decompressed/recompressed merely to move from L1 to L2.
- Small files may be bundled into a newly generated frame when that improves seek/index overhead; large members may span several existing payload frames. The manifest maps TAR/member byte ranges to frames exactly.
- Because reader compatibility is not guaranteed until tested, retain a cold-path compatibility writer that reconstructs the TAR stream and writes a standards-compatible seekable Zstandard file once. Do not double-compress `.zst` members inside an outer `.tar.zst` as the baseline.

10.3 Why frames, not ordinary blocks

A Zstandard frame is independently decodable. Compressed blocks inside a frame may depend on previous blocks and cannot be used as arbitrary random-access boundaries. Therefore the product's "random access compression block size" setting means **uncompressed bytes per independent frame/checkpoint**. Code, metrics, manifests, and tests use the precise term `frame`.

10.4 Batching related logs

The packer selects sealed objects by affinity group so one diagnosis usually opens one or a few archives. Within a group, use deterministic member ordering:

1. Manifest and compact relationship metadata.
2. Root console/stage logs.
3. Downstream/matrix child logs in graph order.
4. Test reports/evidence.
5. Derived indexes/signatures.

The packer may delay sealing briefly to collect late downstream/test data, but has a maximum wait and never blocks interactive reads. Completed members remain readable from L1 until the pack is committed. Oversized groups split into volumes; small adjacent builds may share a pack only when classification, retention, user/profile, and locality policy match.

10.5 Adapter modes

Preferred order:

1. **Direct Rust library/FFI** if the API is stable, memory-safe at the boundary, cancellable, and straightforward to sign/package.
2. **Managed Rust sidecar** over stdin/stdout or a private named pipe with a versioned bounded protocol, opaque IDs, lifecycle management, and no public listener.
3. **Direct non-FUSE archive API** exposed by the project, especially for Windows.
4. **FUSE/WinFsp mount** only as an optional human/diagnostic convenience, never required by the MCP read path.
5. **Native Go fallback** implementing the same seek-table/TAR/manifest contract.

10.6 Ratarmount-rs qualification

Review the exact repository and release candidate for:

- License, provenance, ownership, maintenance/release process, SBOM, dependencies, unsafe Rust, fuzzing, and security policy.
- Supported seekable-Zstd dialect and compatibility with the selected writer.
- TAR edge cases, sparse/long paths, duplicate names, PAX/GNU headers, truncation, corruption, and archive bombs.
- Direct API/sidecar/FUSE modes on Windows x64 first and Linux where gateway deployment is planned.
- Persistent index format, memory mapping, rebuild, migration, cancellation, and mismatched-index detection.
- Concurrent reads, descriptor limits, cache behavior, endpoint-protection impact, and crash recovery.
- 10 GiB, 100 GiB, and at least 1 TiB logical corpus tests with representative line sizes and member counts.

A qualification failure changes the adapter choice, not the on-disk contract or product backlog.

10.7 Transactional packing and recovery

1. Select sealed eligible L1 objects under leases.
2. Prefer assembling a TAR stream from generated header/padding frames plus validated existing L1 payload frames without payload recompression; otherwise use the approved one-time compatibility repack path. Write a temporary seekable `.tar.zst`, manifest, and indexes using resource throttles.
3. Verify all checksums/format invariants and sample members/ranges with both ratarmount-rs and native reader.
4. fsync as required and atomically publish the pack/catalog transaction.
5. Release leases and delete L1 sources only after publish is durable.
6. On crash, recover to either the complete old L1 representation or the complete new L2 representation; never a catalog pointing to partial data.

Indexes are built off the interactive request path. A stale or mismatched index is quarantined/rebuilt; a user read does not synchronously scan a multi-gigabyte pack.

10.8 Performance and bandwidth SLOs

Initial qualification targets, calibrated on corporate Windows hardware and gateway Linux hardware:

- Repeated diagnosis of a sealed build downloads zero duplicate Jenkins log bytes.

- A 64 KiB logical range read decompresses at most the containing frame(s), with target read amplification ≤ 2 frames and ≤ 32 MiB uncompressed at the default setting.
- Warm 64 KiB L2 read < 100 ms p95; cold local-SSD read < 500 ms p95.
- Existing index open < 500 ms p95 per pack or lazy enough not to block the request.
- No single read extracts/decompresses the complete pack.
- Packing/indexing runs below interactive priority and yields under CPU, disk, battery, VPN, or endpoint-protection pressure.
- At least four concurrent readers have bounded RSS and no global serialization.
- Pack target 4-16 GiB physical is adjusted only from measured seek, index, compaction, and failure-domain results.

Metrics record requested bytes, compressed bytes read, frames opened, logical bytes decompressed, archive/member count, index cache state, and wall/CPU time.

11. Enterprise feature inventory

The feature set is divided into launch requirements, post-launch enterprise diagnostics, and optional integrations. "Launch" means production-ready for a limited read-only rollout, not merely present in code.

11.1 Platform and profile management

Launch required

- Versioned profile configuration with enterprise policy overlays.
- Multiple Jenkins controller profiles.
- `profile add/list/show/remove` commands with secret-free output.
- `login/logout/status/doctor` commands.
- OS credential-store integration.
- Import/export of non-secret profile configuration.
- Capability discovery and compatibility report.
- Configuration schema validation and safe migrations.
- Global read-only kill switch from Cursor/profile/signed policy, enforced at registry, dispatch, service, and transport boundaries.
- Optional signed MCP-side RBAC that restricts tools, targets, bytes, cache/export actions, and mutations beyond Jenkins RBAC.
- Local cache status, quota, prune, pin, unpin, verify, and repair commands.
- Automatic data-directory ACL validation.

Post-launch

- Enterprise-managed signed RBAC/read-only policy with subject/group mapping, revocation, explainability, and signature verification.
- AgentCore/managed gateway deployment with per-user downstream credential provider and cache isolation.
- Managed update channel and deployment-ring support.
- Cross-profile read-only search with explicit source labels.
- Offline cached investigation mode.

11.2 Jenkins jobs, folders, and views

Launch required

- List/search jobs with server-side tree selection and local filtering.

- Full folder paths, views, disabled/buildable state, and last-build summaries.
- Job details and parameter definitions without secret defaults.
- Paginated build history.
- Multibranch and pull-request branch discovery where available.
- Matrix parent/child awareness.
- Stable job reference objects rather than requiring the model to compose URLs.
- Permission-denied versus not-found distinction without leaking inaccessible job existence beyond Jenkins behavior.

Post-launch

- Job change/history metadata when available.
- Ownership/contact metadata from conventions or approved plugins.
- Favorite/recent job local shortcuts.

11.3 Builds and queue

Launch required

- Build metadata, result, timing, parameters, causes, actions, and URLs.
- Queue listing and queue item status.
- Running builds and executor placement.
- Adaptive wait with cancellation and bounded duration.
- Build search by result, time, branch, commit, cause, and selected parameters.
- Last successful, last failed, last unstable, and baseline build resolution.
- Accurate freshness/cached status.

Post-launch

- ETA estimates based on recent comparable builds.
- Queue pressure analysis and likely blocking reason.
- Build duration regression detection.

11.4 Console and stage logs

Launch required

- Incremental console log mirror.
- Range, tail, and line-based reads from local storage.
- Literal and safe RE2-compatible regular-expression search.
- Context before/after matches.
- Search cancellation and match limits.
- ANSI/OSC/control-character sanitation for returned text.
- Stable evidence references: profile, job, build, generation, raw range, line range, checksum.
- First meaningful error, last meaningful error, stack-trace grouping, and repeated-line suppression using deterministic heuristics.
- Redaction before MCP output and before optional telemetry.

Post-launch

- Pipeline stage logs and per-node logs where APIs permit.
- Timestamp normalization and merged timelines across sources.
- External pod/agent log adapters.
- Parser plugins for common build systems while retaining raw evidence.
- Historical signature frequency and regression windows.

11.5 Pipeline structure

Launch when Pipeline REST API is present; graceful fallback otherwise

- Stage graph, status, timing, and failed-stage identification.
- Parallel branch representation.
- Stage-level log references.
- Capability detection for Pipeline REST API and plugin version.
- Clear fallback behavior when plugin APIs are missing or inaccessible.

Post-launch

- Node/step summaries with strict response bounds.
- Critical-path and stage-duration regression analysis.
- Correlation between pipeline nodes and downstream builds.

11.6 Tests and quality signals

Post-launch priority

- JUnit summary, suites, cases, failures, skipped, duration, and age.
- Bounded failure detail and stack traces.
- Flaky-test history across a configurable lookback.
- New failure versus known failure classification.
- Test-duration regression.
- Ginkgo and other structured formats through adapters where present.
- Links to Jenkins-native test pages.

11.7 Source control and change correlation

Post-launch priority

- SCM revision(s), branch, repository identity, and change list.
- Changes since last successful or selected baseline build.
- Culpit list as Jenkins reports it, clearly labeled as correlation rather than proof.
- Commit range and modified-file summary.
- Optional approved source-host links without fetching repository content by default.
- Build-to-build change comparison.

11.8 Upstream/downstream build graph

Post-launch priority

- Upstream and downstream relationships from Jenkins causes/actions.
- Pipeline `build`-step relationship discovery where exposed.
- Parameterized Trigger and multijob patterns where exposed.
- Matrix child traversal.
- Cycle detection, maximum depth, maximum nodes, and per-controller concurrency.
- First failing leaf and earliest failure in time.
- Missing-permission nodes represented without leaking data.
- Graph result handles instead of oversized embedded graphs.

11.9 Artifacts

Launch metadata; post-launch content inspection

- Artifact list with path, name, size when known, and content type guess.
- Selective bounded download with streaming and checksum.
- Local quota separate from logs.
- Explicit allowed types and maximum sizes.
- Safe text/JSON/XML inspection.
- Archive inventory without extraction where possible.
- Zip-slip/path traversal prevention.
- Archive bomb limits: file count, expansion ratio, total expanded bytes, nesting depth, and CPU time.
- Temporary workspace isolation and guaranteed cleanup.
- Never execute artifacts.
- Optional L2 archive packing after validation.

11.10 Nodes, executors, controller, and plugins

Post-launch priority

- Node online/offline state, labels, executors, idle/busy, offline cause, and temporary-offline state.
- Queue-to-label demand and executor saturation.
- Controller version and capability snapshot.
- Installed plugin inventory only when the user has permission and policy allows it.
- Required/recommended plugin capability report.
- Disk-space and monitor summaries exposed by Jenkins APIs where authorized.
- Safe controller health overview with no administrative mutations.

11.11 High-level diagnostics

Post-launch priority, central product value

- `diagnose_build`: failed stage, first meaningful failure, test failures, changes since green, downstream failures, repeated signatures, and evidence.
- `compare_builds`: result, stages, durations, parameters, commits, tests, and error signatures.
- `find_regression_window`: first build exhibiting a signature or failed test.
- `trace_failure_graph`: bounded upstream/downstream graph and earliest likely origin.
- `summarize_pipeline`: compact stage/timing/failure representation.
- `find_flaky_tests`: frequency and recency with evidence.
- `explain_queue_delay`: label demand, blockers, and executor state.
- `survey_recent_failures`: bounded cross-job failure signatures.

Every inference must be labeled as heuristic, include confidence, and cite evidence ranges. Diagnostics must not invent a root cause when evidence is ambiguous.

11.12 Mutating operations

Disabled by default

Potentially supported after read-only production maturity:

- Trigger a parameterized build.
- Cancel a queue item.
- Stop a running build.
- Replay/rebuild only where Jenkins APIs and policy make the semantics safe.

Required controls:

- Tool is not registered unless policy enables it.

- Per-profile and per-job allowlists.
- Parameter schema validation and secret-parameter blocking.
- Dry-run/preview response.
- Explicit human confirmation containing profile, job, parameters, and action.
- Correlation ID and local audit event.
- No automatic retries unless idempotency is guaranteed.
- Rate limit and cooldown.
- Jenkins permission check immediately before action.
- Result includes queue/build reference, not a claim of success before Jenkins confirms it.

Administrative actions such as editing job config, credentials, nodes, plugins, scripts, or global settings remain out of scope.

11.13 Optional enterprise integrations

- OpenTelemetry trace/log correlation through approved identifiers.
- External log systems such as Splunk, Elastic, Loki, or cloud logging, using separate per-user connectors and policies.
- Jira/work-item correlation by explicit key, not broad automatic data exfiltration.
- Source host pull-request metadata.
- Incident-management link generation.
- Enterprise policy distribution and fleet health metrics.

Each integration must be a separately installable adapter so it does not increase the core server's permissions or dependency footprint.

12. MCP tool design

12.1 Tool classes

Expose two layers:

Bounded primitives

- `jenkins_list_jobs`
- `jenkins_get_job`
- `jenkins_list_builds`
- `jenkins_get_build`
- `jenkins_list_queue`
- `jenkins_get_pipeline`
- `jenkins_read_log`
- `jenkins_search_log`
- `jenkins_list_tests`
- `jenkins_list_artifacts`
- `jenkins_get_nodes`
- `jenkins_get_capabilities`

Triage operations

- `jenkins_diagnose_build`
- `jenkins_compare_builds`
- `jenkins_trace_failure_graph`
- `jenkins_find_regression_window`
- `jenkins_find_flaky_tests`
- `jenkins_explain_queue_delay`
- `jenkins_survey_recent_failures`

The model should normally call triage operations first. Primitive tools exist for verification and follow-up.

12.2 Contract rules

Every tool contract must include:

- Explicit profile and target references.
- Bounded defaults and maximums.
- Server-enforced pagination.
- A `freshness` option with values such as `cache_ok`, `refresh_if_stale`, and `force_refresh`, with policy limiting force refresh.
- Structured `source` and `evidence` references.
- `truncated`, `continuation`, and byte/count metadata.
- Stable machine-readable error codes.
- Jenkins HTTP status only when safe and useful.
- Capability-related errors that explain which plugin/API is absent.
- No raw URL input for general requests; use typed job/build references.
- No credentials or arbitrary request headers as arguments.
- Tool annotations indicating read-only versus destructive behavior.

12.3 Example diagnosis response

```
{
  "profile": "corp",
  "build": {"job": "products/main", "number": 1842},
  "result": "FAILURE",
  "failed_stage": "integration-tests",
  "assessment": {
    "summary": "Database migration test failed after a permission error.",
    "confidence": "medium",
    "heuristic": true
  },
  "evidence": [
    {
      "kind": "console_log",
      "lines": {"start": 9312, "end": 9344},
      "raw_bytes": {"start": 812341, "end": 816028},
      "excerpt": "...redacted and bounded...",
      "checksum": "sha256:..."
    }
  ],
  "tests": {"new_failures": 1, "known_flaky": 0},
  "changes_since_green": {"commits": 3, "files": 12},
  "downstream": {"visited": 4, "failed": 1},
  "truncated": false,
  "freshness": {"log": "cached-complete", "metadata_age_seconds": 4}
}
```

12.4 Resources and handles

For large cached objects, consider MCP resources or opaque continuation handles. A handle must:

- Be scoped to the local user and profile.
- Contain no secret or filesystem path.
- Expire or remain valid according to documented cache semantics.
- Be unforgeable or validated against local state.
- Never grant access beyond the user's Jenkins permissions and local cache policy.

12.5 Tool authorization metadata

Every tool contract declares side-effect class, required MCP permission, Jenkins permission/capability, target type, network/disk/result budget, cache behavior, and whether user confirmation is required. The registry and policy engine consume the same metadata so aliases or new tools cannot bypass default-deny policy.

13. Diagnostics and search engine

13.1 Search levels

1. **Literal byte/line search:** fastest and default.
2. **RE2-compatible regular expression:** safe from catastrophic backtracking.
3. **Structured event search:** severity, parser, test, stage, source, and signature.
4. **Historical signature search:** across selected jobs/builds within strict limits.
5. **Optional semantic search:** only after a privacy, cost, and accuracy review.

13.2 Deterministic extraction pipeline

Before involving a model, locally compute:

- ANSI/control sanitation.
- Redaction.
- Line boundaries and timestamps.
- Common stack-trace grouping.
- Repeated-line folding with counts.
- Error/warning marker extraction.
- Test failure extraction.
- Build-tool parser outputs.
- Stable signature hashes with volatile values normalized.
- Candidate first-cause ordering.

This improves speed, repeatability, and token use. Raw evidence remains retrievable for verification.

13.3 Prompt-injection handling

Logs and artifacts are untrusted content. The MCP must:

- Mark returned text as untrusted build output.
- Strip terminal control sequences and unsafe hyperlinks.
- Never interpret log text as MCP instructions.
- Avoid tool descriptions that invite the model to follow commands found in logs.
- Prefer structured fields around excerpts.
- Add tests containing malicious instructions, fake system messages, and credential-harvesting text.

13.4 Historical analysis limits

- Default lookback by build count and time.
- Maximum jobs/builds/log bytes scanned per request.
- Local-only search when data is cached; explicit consent/policy for broad remote mirroring.
- Cancellation checked frequently.
- Progress metadata for long operations where the MCP protocol/client supports it.
- Results sorted by relevance and time, with deduplicated signatures.

14. Security and privacy design

14.1 Threat model

Protect against:

- Credential leakage through CLI arguments, environment dumps, logs, crash reports, model context, or support bundles.
- SSRF or credential forwarding through Jenkins-provided URLs and redirects.
- Malicious or compromised Jenkins responses.
- Log/artifact prompt injection.
- Oversized JSON, compressed bodies, logs, and archives causing memory or disk exhaustion.
- Path traversal and archive bombs.
- Unauthorized mutation by an over-eager model.
- Cross-profile cache or identity confusion.
- Local multi-user data exposure.
- Tampered binaries, dependencies, updates, policies, archives, or indexes.
- Stale OAuth sessions after role removal or device compromise.
- Sensitive customer data retained beyond policy.

14.2 Security controls

- Secrets in OS credential stores only.
- Zero secrets in MCP schemas, outputs, logs, metrics, and crash diagnostics.
- Read-only tool registry by default.
- Origin-pinned Jenkins client.
- TLS and optional mTLS/custom CA support.
- Response and decompression bounds.
- Strict JSON decoding with unknown-field policy on configuration.
- Safe regex engine.
- ACL-protected cache and per-profile namespaces.
- Configurable redaction rules plus built-in detectors for common token/password/key formats.
- Redaction applied before model output; optional before persistent cache when policy requires it, with awareness that pre-cache redaction loses forensic fidelity.
- Signed enterprise policy and signed application updates.
- SBOM, provenance, dependency scanning, secret scanning, and vulnerability response process.
- Local audit events without sensitive payloads.
- Cache purge on logout configurable by profile policy.
- OAuth refresh failure and revocation handling that fails closed for remote operations.

14.3 Redaction strategy

Use layered redaction:

1. Built-in exact secret values known to the process, represented as non-reversible matchers and never logged.
2. Structured parameter redaction based on Jenkins parameter names/types and policy patterns.
3. Pattern detectors for bearer tokens, API keys, private keys, connection strings, and credential URLs.
4. Enterprise-configured patterns.
5. Optional PII/customer-data patterns for model output.

Return redaction counts and categories, not matched values. Test false positives so error evidence remains useful.

14.4 Audit events

Record locally, and optionally export through approved telemetry:

- Process/version/profile startup.
- Login/logout and credential source, without token details.
- Jenkins identity confirmation.
- Tool name, target hash, duration, result code, cache status, network bytes, output bytes.
- Mutation preview, confirmation, execution, and Jenkins reference.
- Policy changes, cache purge, archive verification failures, and security warnings.

Audit retention and export are policy-controlled. Do not write full prompts, log excerpts, parameters, or artifact contents by default.

14.5 Supply chain

- Pin Go module versions and verify checksums.
- Review the fork's MIT license and retain attribution.
- Run `govulncheck`, static analysis, license scanning, secret scanning, and dependency review in CI.
- Build reproducibly where practical.
- Sign binaries and release manifests.
- Publish SPDX or CycloneDX SBOM.
- Generate build provenance/attestations.
- Verify any ratarmount-rs binary or library signature/hash before packaging or first use.
- Maintain a documented emergency disable mechanism for vulnerable adapters.

14.6 Authorization-layer security

- Read-only and MCP RBAC are independently tested from Jenkins RBAC.
- Signed enterprise policy can only tighten user configuration.
- Verified identity/group inputs are provenance-labelled; unsigned/user-supplied subjects are rejected.
- Continuation/evidence handles are subject/profile/policy bound to prevent cross-user cache access.
- Gateway caches are partitioned or cryptographically namespaced so two users with different Jenkins visibility cannot receive each other's derived data.
- OAuth-required profiles test fail-closed behavior on every route and cannot silently downgrade to Basic/anonymous/session authentication.

15. Performance engineering plan

15.1 Initial service-level objectives

These are engineering targets for a reference corporate workstation on local SSD and a normally responsive Jenkins controller. Baseline measurements may adjust them, but regressions must remain visible.

Measure	Initial target
Warm process startup to MCP ready, no OAuth interaction	p95 <= 500 ms
Idle resident memory, no archive sidecar	<= 100 MiB
Metadata tool local overhead excluding Jenkins latency	p95 <= 100 ms

Measure	Initial target
Cold metadata call overhead above Jenkins response	p95 <= 250 ms
Cached 64 KiB log read	p95 <= 150 ms
Warm L2 64 KiB archive entry read	p95 <= 100 ms
Cold local-SSD L2 64 KiB archive entry read	p95 <= 500 ms
Literal search of 1 GiB cached log	p95 <= 1.5 s
RE2 search of 1 GiB cached log	p95 <= 4 s for bounded patterns/results
Default tool result	target <= 64 KiB
Absolute tool result	<= 1 MiB
Normal log network duplication	zero duplicate completed ranges
Compression ingest	>= 2x expected network rate or >= 200 MiB/s on reference hardware
Typical text-log compression ratio	>= 3:1 target, explicitly data-dependent
Default total cache quota	10 GiB
Secret-canary leaks	zero

15.2 Default concurrency budgets

Starting values per controller/profile:

- 8 concurrent small metadata requests.
- 2 concurrent log/artifact transfers.
- 1 active compaction worker per user.
- 1 expensive historical search per profile.
- Global CPU worker pool sized from available processors but capped by policy.
- Memory budget for decompressed windows and buffers, with backpressure rather than growth.

Use adaptive behavior based on Jenkins throttling, local CPU/disk pressure, and observed response time. Never allow a model to set unbounded concurrency directly.

15.3 Benchmark corpus

Create synthetic and sanitized representative fixtures:

- 1 MiB, 100 MiB, 1 GiB, and 10 GiB console logs.
- Highly repetitive logs, nearly incompressible logs, very long lines, mixed encodings, ANSI output, and binary contamination.
- Logs that grow while read, truncate, or restart.
- 10, 1,000, and 100,000 jobs/build references.
- Deep folders, multibranch jobs, matrix jobs, and high-fan-out build graphs.
- JUnit files from small to very large.
- Artifact archives with traversal attempts, bombs, and corrupt members.
- L2 packs from 4 GiB to at least 100 GiB physical during qualification.

Benchmark cold and warm OS cache separately. Report CPU time, wall time, peak RSS, allocations, disk bytes, network bytes, compressed size, and p50/p95/p99 latency.

15.4 Performance regression gates

CI or scheduled hardware runners should fail or alert on:

- More than 10% regression in key local p95 latency without approved explanation.
- More than 10% increase in bytes downloaded for fixed fixtures.
- More than 15% increase in peak memory for large-log tests.
- Tool output exceeding contract limits.
- Duplicate progressive log ranges.
- Archive random read degenerating into full scan/decompression.
- Compaction blocking interactive reads beyond the latency budget.

15.5 CPU and allocation strategy

- Stream JSON/log bodies; avoid `io.ReadAll` for unbounded content.
- Reuse bounded buffers through pools only where profiling proves benefit and secrets cannot remain exposed unexpectedly.
- Avoid converting large byte slices to strings repeatedly.
- Keep decompression windows bounded.
- Use content hashing in a streaming pass.
- Build indexes incrementally when possible.
- Separate foreground and background worker priorities.
- Profile with realistic Windows antivirus and endpoint security enabled.

15.6 Initial efficiency budgets

- Completed log bytes fetched from Jenkins: once per log generation unless integrity/retention forces a refetch.
- Metadata calls: coalesced and bounded by tool-specific request budgets.
- MCP result: 64 KiB target, 1 MiB hard ceiling.
- L1/L2 range read: decompress only intersecting independent frame(s); default maximum 32 MiB logical work for a 64 KiB request before an explicit continuation/override.
- Archive diagnosis locality: common build diagnosis should touch one affinity pack plus explicitly traversed downstream volumes.
- Compression/packing CPU: lower scheduling priority than interactive Jenkins/tool work and cancellable/yielding.
- Gateway mode: measure Jenkins-side bytes separately from gateway-to-client MCP bytes so near-source savings are provable.

16. Reliability and data integrity

16.1 Transactional ingestion

A progressive segment is acknowledged only after:

1. The response has passed size and protocol checks.
2. Compressed chunk data is durably written or safely buffered according to policy.
3. Checksum and metadata are recorded.
4. SQLite transaction commits the new logical offset.

After a crash, the recovery process removes orphan temporary files, verifies committed chunks, and resumes from the last committed offset.

16.2 Integrity model

- SHA-256 or approved equivalent for chunks, manifests, packs, and optional downloaded artifacts.
- Fast non-cryptographic checksum may be added for internal corruption detection but not replace the cryptographic checksum.
- Manifest schema version and application version recorded.
- Archive index identity bound to pack size, checksum, and modification identity.
- Sample verification during normal operation; full verification through maintenance command.
- Corruption quarantines the object and triggers a re-fetch only when remote data is still available and policy permits.

16.3 Failure behavior

- Jenkins unavailable: return cached data with age and stale status when the tool permits it.
- OAuth expired/revoked: fail remote calls, retain or purge cache according to policy, and provide a safe re-login status.
- Disk full: stop ingestion, preserve committed state, continue metadata-only or cached reads when safe.
- Corrupt L1 chunk: quarantine, rebuild from Jenkins or L2.
- Corrupt L2 pack/index: do not mount/read it silently; rebuild index or rehydrate from source when possible.
- Sidecar crash: restart with rate limits; fall back to native archive reader when supported.
- Capability/API change: invalidate capability cache and degrade to supported primitives.
- Cancellation: terminate HTTP reads, search workers, decompression, polling, and sidecar calls promptly.

16.4 Schema migration

- Version SQLite schema, chunk headers, manifests, handles, and profile configuration independently.
- Migrations are transactional and have backup/rollback strategy.
- Large cache migrations may run lazily or rebuild derived indexes.
- Never require reading all archived log content merely to upgrade metadata.
- Test upgrade from every supported enterprise release, not only the immediately previous one.

17. Observability and supportability

17.1 Local logging

- Structured logs to a private rotating file or stderr diagnostic mode.
- Default level should not include Jenkins response bodies, parameters, excerpts, filesystem paths containing sensitive names, or authorization data.
- Correlation IDs link MCP call, Jenkins requests, cache work, and audit events.
- Configurable sampling for repetitive success events.
- A support-bundle command that performs explicit privacy scrubbing and previews included categories.

17.2 Metrics

Local metrics should include:

- Tool latency and result codes.
- Jenkins request latency/status family.
- Network and MCP bytes.
- Cache hit ratio.
- Active log mirror lag.

- Compression/decompression throughput and ratio.
- Search throughput.
- L1/L2 size and object count.
- Archive read/index/compaction latency.
- Evictions and quota pressure.
- OAuth refresh success/failure.
- Redaction counts by category.

Central export is opt-in or enterprise-managed and excludes high-cardinality raw job names unless explicitly approved.

17.3 doctor command

`jenkins-mcp doctor --profile corp` should test and report:

- Binary/version/signature state.
- Configuration validity and policy source.
- Data-directory ACL and free space.
- Credential-store accessibility, without retrieving/displaying secret values unnecessarily.
- DNS/TLS/proxy/CA connectivity.
- Authenticated identity.
- Jenkins version and permissions for representative read endpoints.
- Capability/plugin availability.
- Progressive log behavior against an approved small build where configured.
- Archive adapter availability and index directory.
- Cache integrity sample.
- Performance warnings such as cloud-synced cache path or slow storage.

The output must be safe to share after a privacy review.

18. Testing strategy

18.1 Unit tests

- Job path encoding and origin validation.
- Profile and policy validation.
- Credential provider behavior with fake keyrings.
- OAuth state/nonce/PKCE and token validation.
- HTTP redirect, retry, timeout, and body-limit behavior.
- Progressive log offset/generation state machine.
- Chunk boundaries, line indexes, checksums, compression, encryption.
- Search limits, cancellation, context windows, and long lines.
- Redaction and control-character sanitation.
- Tool response budgeting and continuation handles.
- Mutation policy decisions.
- Pack manifests, archive mapping, and recovery journals.

18.2 Jenkins integration tests

Maintain disposable Jenkins test matrices covering:

- Supported Jenkins LTS versions.
- API token auth.

- JWT bearer auth with test issuer.
- Role/folder permissions and inaccessible resources.
- Freestyle, Pipeline, multibranch, pull-request-like, matrix, and parameterized jobs.
- JUnit and artifacts.
- Pipeline REST API present/absent and multiple supported versions.
- Running, queued, cancelled, truncated, and completed logs.
- Reverse proxy path prefixes and redirects.
- Custom CA, proxy, and optional mTLS scenarios.

18.3 Protocol tests

- Official MCP Go SDK conformance suite for supported protocol versions.
- Cursor integration smoke tests for stdio lifecycle, cancellation, tool listing, and large bounded responses.
- Backward-compatibility tests for tool schemas and handles.
- Streamable HTTP security tests only if that mode ships.

18.4 Security tests

- Secret canaries across logs, error paths, metrics, support bundles, and MCP results.
- SSRF and cross-origin redirect attempts.
- Malformed and oversized Jenkins responses.
- OAuth mix-up, replay, wrong issuer/audience, expired tokens, and JWKS rotation.
- Prompt injection fixtures.
- Regex denial-of-service attempts.
- Archive traversal, bombs, symlinks, devices, nested archives, and corruption.
- Unsafe ACL/data path tests.
- Fuzzing for parsers, manifests, log state machine, URL/path handling, and MCP inputs.

18.5 Performance and soak tests

- Multi-hour running build mirror.
- Concurrent searches while compaction runs.
- Repeated process restart and cache recovery.
- OAuth refresh over long sessions.
- Thousands of local MCP requests with cancellation.
- Disk pressure and forced low-space behavior.
- Jenkins throttling, latency, disconnects, and partial responses.
- 100 GiB+ L2 qualification corpus and pack churn.

18.6 Mandatory authorization scenarios

- User has Jenkins administrator rights but MCP role permits only log/test reads: trigger/config/artifact-download operations are denied locally before Jenkins.
- Cursor starts with `--read-only` while profile allows mutations: no mutation tool is listed and direct invocation is denied.
- Signed enterprise policy forces read-only while user removes the CLI flag: read-only remains active.
- OAuth token is valid but presented to a progressive-text or artifact route outside the default API pattern: route is protected and identity is attributed correctly.
- Invalid bearer token cannot fall through to another Jenkins authentication mechanism in an OAuth-required profile.

- Two gateway users with different job visibility never share cached derived results or archive handles.

19. Packaging, deployment, and lifecycle

19.1 Windows-first packaging

- Signed standalone `.exe` and optionally signed MSI/MSIX according to corporate deployment standards.
- Default data under a private per-user local application-data directory.
- No administrator rights for normal installation or operation unless company deployment requires them.
- Windows Credential Manager integration.
- Windows Event Log optional for sanitized security events; private rotating logs otherwise.
- Long-path support and endpoint-security compatibility testing.
- Cursor configuration references executable and profile only, never credentials.

Example secret-free Cursor configuration:

```
{
  "mcpServers": {
    "jenkins": {
      "command": "C:\\Program Files\\JenkinsMCP\\jenkins-mcp.exe",
      "args": ["serve", "--profile", "corp", "--stdio"]
    }
  }
}
```

19.2 Linux and macOS

- Native binaries and OS keyring integrations.
- No FUSE requirement for core operation.
- Package or archive format chosen by enterprise distribution needs.
- Same profile and cache schemas where portability is safe; secrets never exported automatically.

19.3 Updates

- Prefer managed deployment through existing enterprise software distribution.
- If self-update is implemented, use signed metadata, rollback protection, staged channels, and policy control.
- Database migrations run only after binary verification and preflight.
- Preserve last-known-good binary where policy allows.
- Publish release notes with tool-schema, storage-schema, auth, and policy changes.

19.4 Support window

Define:

- Supported Jenkins LTS versions.
- Supported MCP protocol versions and Cursor versions.
- Supported OS versions/architectures.
- Supported OAuth providers or standards profile.
- Supported ratarmount-rs adapter versions and pack schemas.
- Vulnerability response and emergency kill-switch process.

19.5 Optional managed gateway package

Package the same core as a non-root Linux container/service only after the local read-only pilot. Add workload limits, per-user subject binding, AgentCore credential-provider adapter, encrypted/partitioned cache roots, health/readiness, horizontal-concurrency tests, and no public archive sidecar listener. The gateway build must not enable mutations merely because it is centrally hosted.

20. Delivery roadmap

Phase 0 - Baseline and architecture lock

- Fork and pin the Simon baseline; preserve useful behavior with integration fixtures.
- Split packages and add CI/security/performance harnesses.
- Reproduce and fix hidden progressive-log over-download.
- Approve ADRs for no-native-3LO terminology, identity modes, read-only/RBAC, seekable Zstandard format, affinity packs, and ratarmount-rs integration.
- Obtain, pin, code-review, reproduce, and benchmark the exact production `ratarmount-rs` repository and commit/release selected by engineering.

Phase 1 - Secure local read-only pilot

- Windows-first signed local stdio binary.
- Personal API token in Windows Credential Manager.
- Verified Jenkins identity, hardened HTTP, route-safe URL construction.
- Global read-only kill switch configurable from Cursor and forced by policy.
- Restricting MCP-side RBAC policy engine and audit.
- Incremental log mirror, independent Zstandard frames, SQLite line indexes, literal/regex search, redaction, bounded results.
- No L2 dependency required for pilot readiness, but format prototype must exist.

Phase 2 - OAuth resource-server path and seekable L2

- Dedicated Entra Jenkins API resource and local public-client PKCE app.
- Qualify/harden `jwt-auth-filter` or approved alternative.
- Test all MCP-used Jenkins routes, group mapping, revocation, JWKS/key rotation, and anti-fallback.
- Implement seekable multi-frame `.tar.zst`, related-log affinity grouping, ratarmount-rs adapter, native reader, transactional compaction, quota, and repair.

Phase 3 - Enterprise diagnostics

- Pipeline stage/node logs, JUnit/Ginkgo, flaky tests, changes since green, build graph traversal, artifacts, nodes/queue/controller health.
- Evidence-backed `diagnose_build`, comparison, signature extraction, deduplication, and history survey.
- Continuous performance gates for network bytes, frame amplification, memory, index/archive latency, and model output.

Phase 4 - Managed AgentCore/gateway integration

- Confirm AgentCore identity/provider contract and per-user vault semantics.
- Prototype direct Jenkins-audience token and on-behalf-of/RFC 8693 exchange paths.

- Add per-user cache isolation, correlation/audit, rate budgets, and near-source bandwidth measurements.
- Only if required, design a narrow Jenkins token broker/filter extension. A full 3LO authorization-server plugin is a separate project and release gate.

Phase 5 - Controlled mutations and production rollout

- Mutations remain disabled by default and unavailable under global read-only.
- Add allowlisted trigger/cancel/replay with parameter policy, exact preview, bound confirmation, no unsafe retries, and audit.
- Complete independent security review, privacy/retention review, chaos/fuzz/migration tests, documentation, limited pilot, and staged deployment.

21. Release gates

A production release must not ship until all applicable gates pass.

Security gate

- No shared Jenkins identity is required in local or managed mode.
- No token, cookie, authorization header, secret parameter, or refresh material appears in CLI arguments, config, model output, standard logs, or support bundles.
- Read-only is the default and a stronger `true` value cannot be disabled by profile, Cursor arguments, environment, tool input, or model request.
- Disabled mutations are absent from tool discovery and denied again at dispatch, service, and Jenkins request-classification layers.
- MCP-side RBAC demonstrably restricts users who have broader Jenkins permissions and cannot grant a Jenkins-denied resource.
- API-token and OAuth identities map to the expected personal Jenkins principal.
- Every OAuth-used Jenkins route fails closed for missing, malformed, expired, wrong-issuer, or wrong-audience bearer tokens; no unintended Basic/session/anonymous fallback remains.
- Origin/redirect/SSRF, cache ACL/purge, redaction, archive traversal/bomb, dependency, SBOM, signing, and vulnerability tests pass.

Performance gate

- Fixed-size log reads do not download an unrequested suffix beyond documented progressive-fetch granularity.
- Large log acquisition streams directly into bounded Zstandard frames without a complete raw staging copy.
- Repeated reads, searches, and diagnosis use cached generations and report duplicate Jenkins bytes avoided.
- Response, graph, history, concurrency, and network-byte budgets are enforced centrally.
- L2 range reads do not fully extract/decompress packs and meet approved frame/read-amplification targets.
- Zero-recompression promotion is used when qualified; the compatibility path remains off the interactive request path.
- Archive indexing/compaction does not violate interactive latency budgets.

Reliability gate

- Crash recovery, disk-full, cancellation, Jenkins/IdP/JWKS outage, corruption, seek-table/index mismatch, policy reload, and sidecar failure tests pass.
- Config, policy, database, frame, pack, and index migrations/rollback pass.
- No data loss occurs during L1-to-L2 transition under fault injection.
- A native Go reader remains a supported recovery/degraded path.

Compatibility gate

- Supported Jenkins LTS and plugin matrices pass, including every non-`api` route used by progressive logs, artifacts, Pipeline, and mutations.
- The exact JWT filter/proxy release passes claim, group, JWKS, audience, route, and fallback tests.
- Cursor stdio lifecycle, cancellation, and `args/env` read-only configuration pass.
- MCP conformance passes for supported protocol versions.
- Native and ratarmount-rs readers return identical logical bytes/ranges for supported pack fixtures.

Usability gate

- A new user can install, add a profile, choose API-token or OAuth login, verify identity/effective policy, diagnose a failed build, understand a policy denial, inspect cache/archive status, and purge local data.
- `doctor` explains TLS, proxy, permission, OAuth, policy, route-protection, cache, and archive problems without recommending insecure bypasses.
- Every heuristic finding has evidence and uncertainty labeling.

22. Key design decisions to approve

Decision	Recommended default	Reason
Base repository	Fork Simon, preserve behavior with tests, then refactor immediately	Useful seed without inheriting monolithic architecture
Runtime transport	Local stdio	No local daemon/port/auth surface in normal use
Local credentials	Personal Jenkins API token in OS keyring	Matches Jenkins scripted-client model and security's no-shared-account requirement
Local OAuth	External IdP Authorization Code + PKCE producing a Jenkins-audience access token	Browser MFA/Conditional Access without Jenkins acting as authorization server
Jenkins OAuth role	Bearer-token resource server through a qualified filter/proxy	Lowest-complexity delegated API path
AgentCore mode	Per-user OBO/token exchange for a Jenkins-audience token	Resource-specific short-lived identity, no shared account
Full Jenkins 3LO plugin	Deferred, decision-gated contingency	Authorization-server responsibilities are high-risk and usually unnecessary
Effective permission	Jenkins allow AND global mode AND MCP RBAC AND limits	MCP can restrict, never elevate
Default operations	Global read-only <code>true</code>	Safe pilot and emergency kill switch
Cursor control	<code>--read-only</code> and/or <code>JENKINS_MCP_READ_ONLY=true</code> ; most restrictive source wins	Easy explicit testing without policy downgrade

Decision	Recommended default	Reason
Log ingest	Progressive, deduplicated, direct-to-independent-Zstandard-frame	Minimizes network, memory, and disk duplication
Hot storage	SQLite metadata + line-aware immutable Zstandard frames	Fast append, range reads, search, and recovery
Cold storage	Bounded semantic seekable multi-frame .tar.zst affinity packs	Compression, low inode count, and bounded random access
Promotion	Zero-recompression TAR header/payload/padding frame assembly when compatible	Avoids cold-tier CPU and write amplification
Archive access	Qualified ratarmount-rs adapter plus native Go reader	Preferred implementation without dependency lock-in
Search	Literal + RE2 + deterministic parsers before vector search	Fast, safe, explainable, and space efficient
Mutations	Later, exact policy + confirmation + no unsafe retries	Reduces model-driven operational risk

23. Open questions requiring owner decisions

1. What exact `ratarmount-rs` repository, commit/release, license, owner/support model, and API surface is intended? Is Rust FFI, a managed local sidecar, or both acceptable to endpoint security?
2. Which seekable-Zstandard representation does that implementation support: the official seek-table format, concatenated frames with a side index, or another dialect?
3. Which Jenkins LTS and exact `jwt-auth-filter`/proxy versions are approved?
4. Can Entra expose a dedicated Jenkins API resource/scope and issue access tokens with the required audience and claims?
5. Does local OAuth permit refresh tokens on managed endpoints, and what Conditional Access/device requirements apply?
6. For AgentCore, should `Microsoft0auth2` OBO/JWT authorization grant or `Custom0auth2` token exchange be used?
7. Must protected Jenkins API paths be OAuth-only, or may personal API tokens remain an approved fallback?
8. What exact Jenkins route list must the bearer filter/proxy protect, including progressive text, artifacts, Pipeline/JUnit, queue, and future mutations?
9. Which MCP RBAC evaluator/format is approved, and who owns signed policy bundles and emergency deny policy?
10. What job/folder/branch/resource naming patterns and sensitivity labels are stable enough for policy?
11. May logs be cached locally as received, or must selected redaction occur before persistence?
12. Is application-level encryption required in addition to ACLs and full-disk encryption?
13. What frame size, small-member coalescing threshold, pack target, member limit, quota, and retention policy pass representative benchmarks?
14. Should text artifacts join log affinity packs or remain in separate retention/security domains?
15. Is Windows the only launch OS, and are Windows ARM64, Linux gateway, or macOS required?

16. What telemetry may leave a workstation/gateway and how are user/controller/job identities pseudonymized?
17. Who owns Jenkins-side OAuth hardening, Entra configuration, policy authoring, and incident response?
18. What concrete unmet requirement would justify starting the full Jenkins authorization-server plugin epic?

None of these blocks Phase 0. The architecture isolates the decisions behind providers, policy interfaces, and storage formats so measured prototypes can answer them.

24. Source and technology notes

The revision was researched against primary project/vendor documentation available on July 31, 2026.

Jenkins authentication and OAuth

- [Jenkins Remote Access API](#) documents REST-like endpoints and HTTP Basic authentication for secured remote access; Jenkins recommends API tokens for scripted clients.
- The [2021 Jenkins community question on delegated REST API authentication](#) is useful historical corroboration of the same gap, but the release decision is grounded in current Jenkins/plugin behavior and tested capability matrices rather than an old forum answer.
- [OpenID Connect Authentication plugin](#) is a Jenkins browser/security-realm login integration; its non-front-end guidance tells scripted clients to use API tokens.
- [OpenID Connect Provider plugin](#) issues workload identity tokens from Jenkins builds to external services, which is the opposite direction from user-to-Jenkins API delegation.
- [GitHub Authentication plugin](#) is a GitHub-specific security realm, not a generic Entra/Jenkins authorization server.
- [OAuth Credentials plugin](#) supplies extension points for Jenkins plugins to represent OAuth credentials; it is not a remote delegation server.
- [JWT Auth Filter plugin](#) validates externally issued bearer JWTs, configures JWKS/audience/path, and publishes RFC 9728 protected-resource metadata. Metadata can advertise authorization-server/scopes, but production must separately prove scope/claim enforcement, full route coverage, fail-closed behavior, JWKS caching/rotation, and fallback policy.

AgentCore and Entra

- [AgentCore custom OAuth provider configuration](#) permits discovery-based or explicit issuer, authorization, and token endpoints. For Jenkins, those endpoints belong to Entra/the approved authorization server, not Jenkins.
- [AgentCore OAuth token acquisition](#) documents user-federated authorization-code flows plus secure access/refresh-token storage and reuse.
- [AgentCore on-behalf-of token exchange](#) documents downstream OBO/token-exchange modes, including RFC 8693 and RFC 7523-style profiles.
- [AgentCore workload access tokens](#) bind workload and verified end-user identity for credential retrieval; production should prefer the JWT-verified user path.
- [AgentCore Microsoft identity provider](#) documents Entra audience/token-version and exposed-API requirements.
- The supplied engineering notes describe the company's existing per-user AgentCore pattern for Bitbucket/Jira/Confluence. Those product-specific token/VPC details are context, not Jenkins implementation dependencies.

Compression and archive access

- [Ratarmount](#) documents persistent archive indexes and notes that useful Zstandard seeking requires deliberately multi-frame/seekable output; ordinary compressors commonly create one unsuitable frame.
- [Zstandard compression format](#) defines concatenated frames as independently decodable while compressed blocks inside a frame may depend on earlier data.
- [Zstandard Seekable Format](#) defines independent frames followed by a final seek-table skippable frame.
- `'t2sz'` is a reference implementation that maps TAR members or bounded input regions to independent Zstandard frames and supports member grouping/splitting tradeoffs.
- The user-preferred `ratarmount-rs` implementation was not identified by an exact authoritative public repository in the supplied notes or public research. Adoption therefore requires an explicit dependency handoff plus code, license, supply-chain, API, Windows, recovery, format-compatibility, and performance qualification of a pinned revision.

Cursor

- [Cursor MCP documentation](#) supports local stdio server configuration through `command`, `args`, and `env`, which is suitable for an explicit `--read-only`/environment restriction.

25. Bottom line

The engineer notes are largely correct and materially improve the architecture. Jenkins should not be represented as a native general-purpose 3LO authorization server. The preferred OAuth path is an external IdP issuing a Jenkins-audience access token, with Jenkins acting as a qualified bearer-token resource server. AgentCore can acquire that per-user token through authorization-code 3LO and its user/workload-bound vault or through OBO/token exchange. A full Jenkins authorization-server plugin is possible but remains a separate, security-critical contingency after a formal decision gate.

The enterprise MCP must start local, personal, and read-only. A global kill switch is enforced below tool discovery, and optional MCP RBAC can further restrict tools, Jenkins resources, arguments, caching, and volume without ever granting access Jenkins denied.

For performance, progressive log bytes are deduplicated and compressed while streaming into independent Zstandard frames. Related sealed logs are grouped into bounded semantic seekable `.tar.zst` packs. The preferred packer reuses existing compressed payload frames, surrounding them with small generated TAR header/padding frames, so promotion does not recompress log data when selected readers support it; a native Go reader and compatibility writer prevent lock-in. The exact engineering-selected `ratarmount-rs` dependency remains the preferred L2 adapter once it has been supplied, audited, reproduced, and benchmarked.

The companion agent backlog expresses these decisions as dependency-aware implementation tasks, including OAuth capability tests, JWT filter hardening, AgentCore 3LO/OBO, read-only/RBAC enforcement, HTTP wire-compression measurement, seekable archive format, zero-recompression promotion, semantic batching, and a conditional Jenkins authorization-server plugin epic.

End of document